

Midterm Project

Real-Time Data Collection and Stream Processing System

AITU Classroom Environmental & Energy Monitoring

Galymzhan Tamirlan Galymzhanuly
Student ID: 240542 | Group: BDA-2407
Astana IT University
Instructor: Serek Azamat

October 2026

Contents

1	Introduction	2
2	Data Scenario and Stream Architecture	2
2.1	The Room and Sensor Setup	2
2.2	Data Schema	2
2.3	Stream Phases	2
3	Implementation	3
3.1	Stream Generator (<code>stream_generator.py</code>)	3
3.2	Stream Processor (<code>stream_processor.py</code>)	3
3.2.1	Step 1: Online Imputation	3
3.2.2	Step 2: Rolling Statistics	4
3.2.3	Step 3: Derived Indicators	4
3.2.4	Step 4: Anomaly Detection	4
4	Results	4
4.1	Dashboard Figure	5
5	Discussion	5
5.1	What Worked Well	5
5.2	Limitations	5
5.3	What I Would Do Differently	5
6	Conclusion	6

1 Introduction

For this midterm I decided to build a real-time stream processing system for monitoring a classroom at AITU. The idea came from Assignment 2, where we worked with a small batch of classroom sensor data from Room C1.3.101. I wanted to take that dataset further and simulate what it would look like if those sensors were streaming data continuously during a whole lecture.

The goal of the system is to process incoming sensor packets one by one, compute real-time statistics like rolling averages, and automatically raise alerts when something unusual happens — bad air quality, the room getting too hot, or an electrical surge.

I tried to make everything reproducible, so the stream generator uses a fixed random seed (42) and always produces the same 60-minute sequence.

2 Data Scenario and Stream Architecture

2.1 The Room and Sensor Setup

Room C1.3.101 is a lecture and lab space at AITU with an official capacity of 40 students. The room has four types of sensors:

- **Temperature sensor** (Temp_C) — measures indoor air temperature in °C
- **CO₂ sensor** (CO2_ppm) — measures air quality in parts per million
- **Occupancy camera** (Occupancy) — counts heads using computer vision
- **Smart power meter** (Power_kW) — measures total electrical load

Each sensor sends one reading per minute, timestamped at the minute boundary. In a real deployment these would go through an MQTT broker or WebSocket connection. In my simulation I use a Python generator that yields them one at a time.

2.2 Data Schema

Table 1: Incoming sensor packet schema

Field	Type	Units	Normal Range
Time	string	HH:MM	10:00 – 10:59
Room	string	—	C1.3.101
Temp_C	float	°C	20.0 – 24.0
CO2_ppm	float	ppm	400 – 1000
Occupancy	integer	persons	0 – 40
Power_kW	float	kW	1.5 – 7.0

2.3 Stream Phases

I structured the 60-minute stream in phases so that several interesting anomalies appear naturally:

1. **10:00–10:11** — Assignment 2 baseline (12 real records, includes 3 sensor dropouts)
2. **10:12–10:25** — Lecture at full capacity, stable readings
3. **10:26–10:35** — Ventilation failure, CO₂ spikes past 1000 ppm
4. **10:36–10:45** — Thermal accumulation, temperature crosses 24°C
5. **10:46–10:50** — Lab practical, electrical surge above 6 kW
6. **10:51–10:59** — Lecture ends, students leave, everything drops

3 Implementation

3.1 Stream Generator (`stream_generator.py`)

I wrote a `generate_stream()` function that starts with the Assignment 2 baseline records and then continues with simulated data for the remaining 48 minutes. The function takes a `seed` parameter so the output is always the same.

The three intentional sensor dropouts (missing values) appear at:

- 10:02 — `Temp_C` missing
- 10:04 — `CO2_ppm` missing
- 10:07 — `Occupancy` missing

Plus three more dropouts I inject in the extended part (at minutes 18, 33, and 44).

3.2 Stream Processor (`stream_processor.py`)

The `StreamProcessor` class is the core of the project. It has a `process(record)` method that I call for each incoming packet. Inside, it does the following steps:

3.2.1 Step 1: Online Imputation

Missing values are handled before any calculation. I use two strategies:

- For continuous sensors (`Temp_C`, `CO2_ppm`): linear extrapolation from the last two buffered values if available, otherwise forward-fill.
- For discrete sensors (`Occupancy`): always forward-fill from the last known head-count.

Any record with a missing value gets a `SENSOR_DROPOUT` flag.

3.2.2 Step 2: Rolling Statistics

I keep a sliding window (size 5) for each sensor using Python's `deque(maxlen=5)`. For each record I compute:

$$\bar{T}_t = \frac{1}{5} \sum_{i=t-4}^t T_i \quad (\text{rolling mean temperature})$$

$$\bar{C}_t = \frac{1}{5} \sum_{i=t-4}^t C_i \quad (\text{rolling mean CO}_2)$$

$$\bar{P}_t = \frac{1}{5} \sum_{i=t-4}^t P_i \quad (\text{rolling mean power})$$

$$\sigma_T^{(t)} = \text{std}(T_{t-4} \dots T_t) \quad (\text{rolling temperature volatility})$$

3.2.3 Step 3: Derived Indicators

- **Occupancy density:** $d_t = \text{Occupancy}_t / 40$
- **Energy per occupant:** $e_t = P_t / \max(\text{Occupancy}_t, 1)$
- **Temperature delta:** $\Delta T_t = T_t - T_{t-1}$
- **Comfort flag:** True if $20 \leq T_t \leq 24$ and $\text{CO}_{2,t} \leq 1000$

3.2.4 Step 4: Anomaly Detection

I check six conditions after each record is processed:

Table 2: Anomaly detection rules

Alert	Condition
SENSOR_DROPOUT	Any sensor field is None (missing)
CO2_HIGH	$\text{CO}_2 > 1000$ ppm
CO2_CRITICAL	$\text{CO}_2 > 1200$ ppm
OVERHEATING	Temp $> 24.0^\circ\text{C}$
UNDERHEATING	Temp $< 20.0^\circ\text{C}$
POWER_SURGE	Power > 6.0 kW
RAPID_OCC_CHANGE	$ \Delta \text{Occupancy} \geq 6$ persons/min

4 Results

After processing all 60 records I got the following statistics:

- **96 anomaly events** detected across 60 records
- **30 OVERHEATING** events (10:36–10:59 mostly)
- **28 POWER_SURGE** events (10:46–10:59)
- **20 CO2_CRITICAL** events (10:32–10:45 range)

- **10 CO2_HIGH** events (10:26–10:31)
- **6 SENSOR_DROPOUT** events
- **2 RAPID_OCC_CHANGE** events (when students leave)

The most critical situation was around 10:32–10:35, when the room had simultaneously high CO₂ (critical level), temperature above 24°C, and power near the surge threshold.

4.1 Dashboard Figure

The figure generated by my notebook (`stream_dashboard.png`) shows three subplots sharing a common time axis:

1. Temperature and CO₂ raw values with 5-step rolling means and threshold lines
2. Power consumption and occupancy over time
3. Anomaly timeline showing when each alert type fired

5 Discussion

5.1 What Worked Well

The online imputation worked better than I expected. Because I use linear extrapolation from the rolling buffer rather than just forward-filling, the imputed values follow the trend of the data pretty closely. For example, when the temperature sensor dropped out at 10:02, the imputed value (21.8°C) was very close to what would have been recorded.

The anomaly detection is also pretty clean. Because I check multiple conditions per record and log each one separately, I can see clearly in the timeline how different alert types overlap — for example, during the ventilation failure event you can see CO₂_HIGH turning into CO₂_CRITICAL as the concentration builds up.

5.2 Limitations

The main limitation is that my stream is simulated. In a real deployment you'd want to connect to actual sensor hardware. Also, my anomaly thresholds are static — a smarter system would learn the normal range adaptively for each room.

Another thing I didn't have time for is sending alerts somewhere useful, like a notification to the building management system. Right now it all just gets logged to a DataFrame.

5.3 What I Would Do Differently

If I had more time I would add:

- Adaptive thresholds using the rolling standard deviation
- A proper logging system with timestamps and severity levels
- A simple web dashboard using Flask to show the live stream

6 Conclusion

I built a working real-time stream processing system that takes classroom sensor data one record at a time and produces live rolling statistics, derived comfort indicators, and rule-based alerts.

The key thing I learned is how different stream processing is from batch processing. In batch mode I can sort, fill, and aggregate the entire dataset at once. In stream mode I can only use what I've already seen, which forces you to think carefully about state management and what information to keep in memory.

Using the Assignment 2 dataset as the baseline made it easy to validate the output — I could compare the first 12 processed records against the values I computed by hand in Assignment 2 and confirm they matched.